

# Assignment – 2

---

- **Overview of C programming**

## **History of C language**

- C programming is developed by Dennis Ritchie at Bell lab in the early of 1970s. C gained popularity due to its efficiency, portability, and versatility, becoming a foundational language for various applications and influencing other language like C++ and Java. It evolved from the B language and was initially designed for system programming, particularly for rewriting the Unix operating system.
- In 1972 Dennis Ritchie develops C at bell labs as an evolution of the B language, incorporating data types and structures.
- In 1973 the Unix operating system is rewritten in C, solidifying its role in system programming.
- In 1978 Kernighan and Ritchie publish “The C programming language”, defining the syntax and libraries, becoming a widely used references.
- In 1989 ANSI C (C89) is standardized, ensuring portability and consistency across platforms.
- In 1990 ISO/IEC 9899:1990 established C as an international standard.
- C99,C11,C18 subsequent revisions introduce new features and enhancements to the language.

## **Evolution of C programming**

K&R C : The initial version of C defined by Kernighan and Ritchie in their book.

ANSI C (C89) : The first official standard for C, ensuring consistency and portability.

ISO C (ISO/ICE 9899:1990) : The international standard for C, further solidifying its global adoption.

C99, C11, C18 : Ongoing updates and standards that have introduced new features and improvement.

### **Impact**

Foundation for other languages :

C's syntax and features have served as a blueprint for numerous programming languages, including C++, Java and Python.

Ubiquitous in system programming :

C remains a dominant choice for low level system programming, including operating system and embedded system.

Efficiency and versatility :

C's efficiency and versatility have made it a popular choice for various applications, from game development to scientific computing.

## **Setting up environment**

To install a C compiler and set up Dev C++ or Visual Studio Code, you will need to install a C compiler like MinGW and then configure

your chosen IDE. For Dev C++, simply follow the installer prompt, while for VS code, you will need to install the C/C++ extension and set up the build tool.

- VS Code's C/C++ extension provides features like syntax highlighting, code completion, and debugging support.
- Dev C++ is a dedicated IDE that often includes the compiler and the other development.

## **Steps to download C compiler**

Dev C++ Installation :

Download Dev C++ : Locate the setup file from the official websites, such as [bloodshed.net](http://bloodshed.net).

Run the setup file : Double click the setup file to begin the installation process.

Follow the wizard : Follow the prompts in the installation wizard, including choosing installation components, location, and libraries.

Complete installation : Once the wizard finishes, Dev C++ is ready to use.

VS Code setup :

Install VS code : Download and install VS Code from the official website.

Install C/C++ Extension : Open VS Code, go to the extension view, and install the C/C++ extension by Microsoft.

Install MinGW : Download and install MinGW-w64 from the MinGW website.

Configure path : Set the environment path for MinGW by adding the bin directory to your system's PATH variable.

Start coding : You are now ready to write C code in VS code and use MinGW to compile and run your programs.

- **Basic structure of C**

The basic structure of C program is divided into 6 parts which makes it easy to read, modify, document, and understand in a particular format.

## **Sections of the C program**

### **Documentation :**

The section consists of the description of the program, the name of the program, and the creation date and time of the program. It is specified at the start of the program in the form of comments. Documentation can be represented as:

Example: `// description, name of the program, programmer name, date, time etc.`

Anything written as comment will be treated as documentation of the program and this will not interfere with the given code.

### **Preprocessor section :**

All the header files of the program will be declared in the preprocessor section of the program. Header files help us to access other's improved code into our code. A copy of these multiple files is inserted into our program before the process of compilation.

Example: `#include<stdio.h>`

`#include<math.h>`

### **Definition:**

Preprocessors are the programs that process our source code before the process of compilation. There are multiple steps which are involved in the writing and execution of the program. Preprocessor directives start with the '#' symbol. The #define preprocessor is used to create a constant throughout the program. Whenever this name is encountered by the compiler, it is replaced by the actual piece of defined code.

Example : #define long long 11

### **Global declaration :**

The global declaration section contains global variable, function declaration and functions which are declared in this scope can be used anywhere in the program.

Example: int num = 18;

### **Main() function :**

Every C program must have a main function. The main() function of the program is written in the section. Operation like declaration and execution are performed inside the curly braces of the main() function. It can be int as well as void too. Void() main tells the compiler that the program will not return any value. The int main() tells the compiler that the program will return an integer value.

Example: void main()

### **Sub programs :**

User defined functions are called in this section of the program. The control of the program is shifted to the called function whenever they are called from the main or outside the main() function. These are specified as per requirement of the programmer.

Example: int sum(int x, int y)

{

```
Return x+y;
```

```
}
```

## Headers in C :

In C programming, header files are essential for code organization and reusability. They serve as a central location for declarations of functions, variables, and macros that can be shared across multiple source files. Header files typically have a `.h` extension.

Purpose of Header Files:

- **Declaration Sharing:**

Header files declare functions, variables, and macros. This allows multiple source files to use these elements without redeclaring them, which prevents errors and improves code maintainability.

- **Code Reusability:**

By placing common declarations in header files, you can reuse them across different parts of a project.

- **Modularity:**

Header files help to break down a program into smaller, manageable modules.

- **Interface Definition:**

They define the interface of a module by specifying what functions and data are available to the rest of the program.

Types of Header Files:

- **Standard Library Headers:**

These are provided by the C standard library and contain declarations for standard functions and macros. Examples include:

- `stdio.h`: Input/output operations (e.g., `printf`, `scanf`).

- `stdlib.h`: General utilities (e.g., memory allocation, random number generation).
- `string.h`: String manipulation functions (e.g., `strcpy`, `strlen`).
- `math.h`: Mathematical functions (e.g., `sin`, `cos`).
- `time.h`: Time and date functions.
- `ctype.h`: Character handling functions (e.g., `isalpha`, `isdigit`).

- **User-Defined Headers:**

These are created by the programmer to organize their code. They are typically used to declare functions and data structures specific to a particular project.

### Including Header Files:

- Header files are included in a source file using the `#include` preprocessor directive.
- **System Header Files:** Included using angle brackets (`<>`), e.g., `#include <stdio.h>`. The compiler searches for these files in standard system directories.
- **User-Defined Header Files:** Included using double quotes (`" "`), e.g., `#include "myheader.h"`. The compiler searches for these files in the same directory as the source file, and then in standard system directories.

### Best Practices:

- Only declarations should be in header files, not definitions (except for inline functions and constants).
- Include guards (`#ifndef`, `#define`, `#endif`) should be used to prevent multiple inclusions of the same header file.
- Header files should be well-documented to explain the purpose of the declared elements.

Example:

```

C
#ifndef MYHEADER_H
#define MYHEADER_H

int add(int a, int b);

#endif

C
#include <stdio.h>
#include "myheader.h"

int main()
{
    int result = add(5, 3);
    printf("Result: %d\n", result);
    return 0;
}

C
#include "myheader.h"

int add(int a, int b)
{
    return a + b;
}

```

## Main() Functions :

In C programming, the `main` function serves as the entry point for program execution. It is a special function that the operating system calls to start the program. Here's a breakdown of its key aspects:

Syntax:

The `main` function can have several valid signatures:



- `int main(void)`: This is the simplest form, indicating that the function doesn't receive any command-line arguments and returns an integer value.
- `int main(int argc, char *argv[])`: This form allows the program to receive command-line arguments.
  - `argc` is an integer that represents the number of arguments passed to the program.
  - `argv` is an array of strings, where each string represents a command-line argument.
- `int main(int argc, char *argv[], char *envp[])`: This form includes an additional parameter, `envp`, which is an array of strings representing the environment variables.

Example:

```
C
#include <stdio.h>

int main() {
    printf("Hello, world!\n");
    return 0;
}
```

In this example:

- `int main()` declares the main function, which returns an integer.
- `printf` is a function that prints a string to the console.
- `return 0` indicates successful program execution.

## Comments :

Comments in C programs are explanatory notes embedded within the code that are ignored by the compiler. They serve to enhance code readability and understanding for developers.

There are two primary types of comments in C:

- Single-line comments: These begin with `//` and continue until the end of the line. They are typically used for short explanations or notes.

```
int x = 10; // Initialization of x
```

- Multi-line comments: These are enclosed within `/*` and `*/`. They can span multiple lines and are suitable for longer explanations or for temporarily disabling blocks of code.

```
C
int y = 20;
```

## Data type :

Each variable in C has an associated data type. It specifies the type of data that the variable can store like integer, character, floating, double, etc.

### Example:

```
int number;
```

The above statement declares a variable with name **number** that can store **integer** values.

C is a statically type language where each variable's type must be specified at the declaration and once specified, it cannot be changed. This is because each data type requires different amounts of memory and allows type specific operations.

### The data types in C can be classified as follows:

| Types                       | Description                                                                                      | Data Types                     |
|-----------------------------|--------------------------------------------------------------------------------------------------|--------------------------------|
| <b>Primitive Data Types</b> | Primitive data types are the most basic data types that are used for representing simple values. | int, char, float, double, void |

| Types                                          | Description                                                                                                     | Data Types                |
|------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|---------------------------|
| <u><a href="#">Derived Types</a></u>           | The data types that are derived from the primitive or built-in datatypes are referred to as Derived Data Types. | array, pointers, function |
| <u><a href="#">User Defined Data Types</a></u> | The user-defined data types are defined by the user himself.                                                    | structure, union, enum    |

In this article, we will discuss the basic (primary) data types in C.

## Table of Content

- [Integer Data Type](#)
- [Character Data Type](#)
- [Float Data Type](#)
- [Double Data Type](#)
- [Void Data Type](#)

Integer Data Type:

The integer datatype in C is used to store the integer numbers (any number including positive, negative and zero without decimal part). Octal values, hexadecimal values, and decimal values can also be stored in int data type in C.

- **Range:** -2,147,483,648 to 2,147,483,647
- **Size:** 4 bytes
- **Format Specifier:** %d

[Format specifiers](#) are the symbols that are used for printing and scanning values of given data types.

**Example:**

We use [int keyword](#) to declare the integer variable:

```
int val;
```

We can store the integer values (literals) in this variable.



```
#include <stdio.h>
```

```
int main() {  
    int var = 22;  
  
    printf("var = %d", var);  
    return 0;  
}
```

A variable of given data type can only contains the values of the same type. So, **var** can only store numbers, not text or anything else. The integer data type can also be used as:

1. **unsigned int:** It can store the data values from zero to positive numbers, but it can't store negative values
2. **short int:** It is lesser in size than the int by 2 bytes so can only store values from -32,768 to 32,767.
3. **long int:** Larger version of the int datatype so can store values greater than int.
4. **unsigned short int:** Similar in relationship with short int as unsigned int with int.

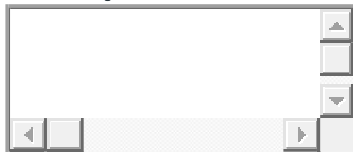
**Note:** The size of an integer data type is compiler dependent. We can use **sizeof operator** to check the actual size of any data type. In this article, we are discussing the sizes according to 64-bit compilers.

### Character Data Type:

character data type allows its variable to store only a single character. The size of the character is **1 byte**. It is the most basic data type in C. It stores a single character and requires a single byte of memory in almost all compilers.

- **Range:** (-128 to 127) or (0 to 255)
- **Size:** 1 byte
- **Format Specifier:** %c

**Example:**



```
#include <stdio.h>
```

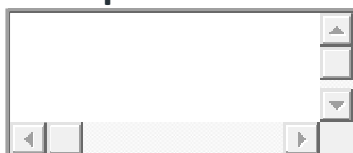
```
int main() {  
    char ch = 'A';  
  
    printf("ch = %c", ch);  
    return 0;  
}
```

**Float Data Type:**

In C programming, float data type is used to store single precision floating-point values. These values are decimal and exponential numbers.

- **Range:** 1.2E-38 to 3.4E+38
- **Size:** 4 bytes
- **Format Specifier:** %f

**Example:**



```
#include <stdio.h>
```

```
int main() {
```

```
float val = 12.45;

printf("val = %f", val);
return 0;
}
```

### Double Data Type:

The double data type in C is used to store decimal numbers (numbers with floating point values) with double precision. It can easily accommodate about 16 to 17 digits after or before a decimal point.

- **Range:** 1.7E-308 to 1.7E+308
- **Size:** 8 bytes
- **Format Specifier:** %lf

### Example:

```
#include <stdio.h>

int main() {
    double val = 1.4521;

    printf("val = %d", val);
    return 0;
}
```

### Void Data Type:

The void data type in C is used to indicate the absence of a value. Variables of void data type are not allowed. It can only be used for pointers and function return type and parameters.

### Example:

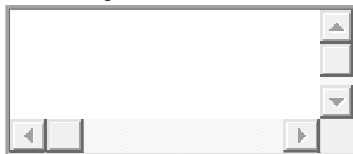
```
void fun(int a, int b){
    // function body
}
```

where function **fun** is a void type of function means it doesn't return any value.

### Size of Data Types in C:

The size of the data types in C is dependent on the size of the architecture, so we cannot define the universal size of the data types. For that, the C language provides the **sizeof()** operator to check the size of the data types.

#### Example



```
int main(){

    // Use sizeof() to know size
    // the data types
    printf("The size of int: %d\n",
        sizeof(int));
    printf("The size of char: %d\n",
        sizeof(char));
    printf("The size of float: %d\n",
        sizeof(float));
    printf("The size of double: %d",
        sizeof(double));

    return 0;
}
```

#### Output

```
The size of int: 4
The size of char: 1
The size of float: 4
The size of double: 8
```

Different data types also have different ranges up to which can vary from compiler to compiler. Below is a list of ranges along with the memory requirement and format specifiers on the **64-bit GCC compiler**.

| <b>Data Type</b>              | <b>Size<br/>(bytes)</b> | <b>Range</b>                    | <b>Format<br/>Specifier</b> |
|-------------------------------|-------------------------|---------------------------------|-----------------------------|
| <b>short int</b>              | 2                       | -32,768 to 32,767               | <b>%hd</b>                  |
| <b>unsigned<br/>short int</b> | 2                       | 0 to 65,535                     | <b>%hu</b>                  |
| <b>unsigned<br/>int</b>       | 4                       | 0 to 4,294,967,295              | <b>%u</b>                   |
| <b>int</b>                    | 4                       | -2,147,483,648<br>2,147,483,647 | to<br><b>%d</b>             |
| <b>long int</b>               | 4                       | -2,147,483,648<br>2,147,483,647 | to<br><b>%ld</b>            |
| <b>unsigned<br/>long int</b>  | 4                       | 0 to 4,294,967,295              | <b>%lu</b>                  |
| <b>long long<br/>int</b>      | 8                       | $-(2^{63})$ to $(2^{63})-1$     | <b>%lld</b>                 |



| Data Type              | Size (bytes) | Range                           | Format Specifier |
|------------------------|--------------|---------------------------------|------------------|
| unsigned long long int | 8            | 0 to 18,446,744,073,709,551,615 | %llu             |
| signed char            | 1            | -128 to 127                     | %c               |
| unsigned char          | 1            | 0 to 255                        | %c               |
| float                  | 4            | 1.2E-38 to 3.4E+38              | %f               |
| double                 | 8            | 1.7E-308 to 1.7E+308            | %lf              |
| long double            | 16           | 3.4E-4932 to 1.1E+4932          | %Lf              |

## Variables in C :

In C programming, a variable is a named storage location in the computer's memory used to hold data. It acts as a container for values that can change during program execution.

Key Characteristics of Variables:

- **Name:**  
Each variable must have a unique name (identifier) that follows specific rules.
- **Data Type:**

Every variable is associated with a data type that determines the kind of value it can store (e.g., integer, floating-point number, character).

- **Value:**

The actual data stored in the variable's memory location. This value can be modified during the program's execution.

### How to Declare a Variable:

To declare a variable, you specify its data type followed by its name. For example:

```
C
int age;

float price;
char initial; // Declares a character variable named 'initial'
```

### How to Initialize a Variable:

You can assign an initial value to a variable during its declaration or later in the code:

```
C
int age = 25;
float price;
price = 19.99; // Assigns 19.99 to the 'price' variable
```

### Types of Variables

- **Local variables:** declared inside a function or block of code, accessible only within that scope.
- **Global variables:** declared outside of any function, accessible from any part of the code.

### Rules for Naming Variables:

- Variable names can consist of letters, digits, and underscores.
- The first character must be a letter or an underscore.
- Case sensitivity: age and Age are considered different variables.

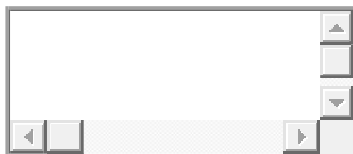
- Reserved keywords (e.g., int, float, if, for) cannot be used as variable names.

Variables are fundamental building blocks in C programming, allowing you to store and manipulate data effectively.

- **Control flow statement :**

In C, programs can choose which part of the code to execute based on some condition. This ability is called **decision making** and the statements used for it are called **conditional statements**. These statements evaluate one or more conditions and make the decision whether to execute a block of code or not.

For example, consider that there is a show that starts only when certain number of people are present in the audience. So, you can write a program like as shown:



```
#include <stdio.h>
```

```
int main() {
```

```
    // Number of people in the audience
```

```
    int num = 100;
```

```
    // Conditional code inside decision making statement
```

```
    if (num > 50) {
```

```
        printf("Start the show");
```

```
    }
```

```
    return 0;
}
```

## Output

Start the show

In the above program, the show only starts when the number of people is greater than **50**. It is specified in the **if statement** (a type of conditional statement) as a condition (**num > 50**). You can decrease the value of **num** to less than **50** and try rerunning the code.

## Types of Conditional Statements in C:

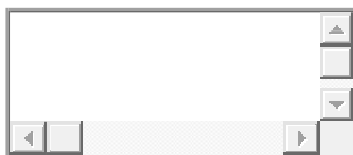
In the above program, we have used if statement, but there are many different types of conditional statements available in C language:

### 1. if in C:

The if statement is the simplest decision-making statement. It is used to decide whether a certain statement or block of statements will be executed or not i.e if a certain condition is true then a block of statements is executed otherwise not.

A **condition** is any expression that evaluates to either a true or false (or values convertible to true or false).

Example



```
#include <stdio.h>
```

```
int main() {
```

```

int i = 10;

// If statement
if (i < 18) {
    printf("Eligible for vote");
}
}

```

## Output

Eligible for vote

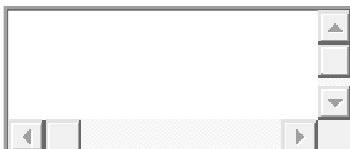
The expression inside **() parenthesis** is the **condition** and set of statements inside **{ } braces** is its **body**. If the condition is true, only then the body will be executed.

*If there is only a single statement in the body, **{ } braces** can be omitted.*

### 1. if-else in C:

- The **if** statement alone tells us that if a condition is true, it will execute a block of statements and if the condition is false, it won't. But what if we want to do something else when the condition is false? Here comes the C **else** statement. We can use the **else** statement with the **if** statement to execute a block of code when the condition is false. The if-else statement consists of two blocks, one for false expression and one for true expression.

- Example



- `#include <stdio.h>`
- 
- `int main() {`

- `int i = 10;`
- 
- `if (i > 18) {`
- `printf("Eligible for vote");`
- `}`
- `else {`
- `printf("Not Eligible for vote");`
- `}`
- `return 0;`
- `}`

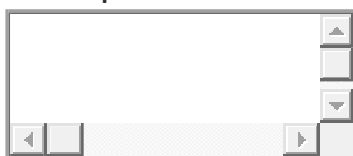
### Output

- Not Eligible for vote
- The block of code following the *else* statement is executed as the condition present in the *if* statement is false.

## 4. switch Statement in C:

The switch case statement is an alternative to the if else if ladder that can be used to execute the conditional code based on the value of the variable specified in the switch statement. The switch block consists of cases to be executed based on the value of the switch variable.

Example



```
#include <stdio.h>
```

```
int main() {
```

```
    // variable to be used in switch statement
```

```
    int var = 18;
```

```
    // declaring switch cases
```

```
    switch (var) {
```

```

case 15:
    printf("You are a kid");
    break;
case 18:
    printf("Eligible for vote");
    break;
default:
    printf("Default Case is executed");
    break;
}

return 0;
}

```

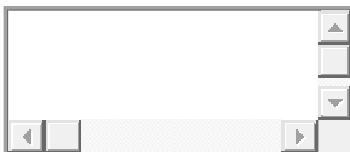
## Output

Eligible for vote

### 2. Nested if-else in C:

- A nested if in C is an if statement that is the target of another if statement. Nested if statements mean an if statement inside another if statement. Yes, C allow us to nested if statements within if statements, i.e, we can place an if statement inside another if statement.

- Example



- `#include <stdio.h>`
- 
- `int main(){`
- `int i = 10;`
- 
- `if (i == 10) {`
- `if (i < 18)`

- `printf("Still not eligible for vote");`
- `else`
- `printf("Eligible for vote\n");`
- `}`
- `else {`
- `if (i == 20) {`
- `if (i < 22)`
- `printf("i is smaller than 22 too\n");`
- `else`
- `printf("i is greater than 25");`
- `}`
- `}`
- `return 0;`
- `}`
- 

### Output

- Still not eligible for vote

### • Looping in C :

- **For Loop, While Loop, and Do-While Loop** are different [loops in programming](#). A For loop is used when the number of iterations is known. A While loop runs as long as a condition is true. A Do-While loop runs at least once and then continues if a condition is true.

### • For Loop in Programming:

- The for loop is used when you know in advance how many times you want to execute the block of code.
- It iterates over a sequence (e.g., a list, tuple, string, or range) and executes the block of code for each item in the sequence.
- The loop variable (variable) takes the value of each item in the sequence during each iteration.

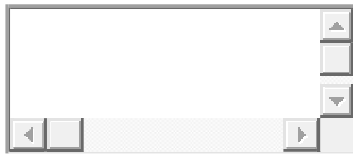


For Loop Syntax:

```
for      (initialization;      condition;      increment/decrement)  {  
  //      Code      to      be      executed      repeatedly  
}
```

Examples:

C++JavaPython3C#JavaScript



```
#include <iostream>  
using namespace std;
```

```
int main()  
{  
    for (int i = 0; i < 5; i++)  
        cout << i << "\n";  
    return 0;  
}
```

### While Loop in Programming:

- The while loop is used when you don't know in advance how many times you want to execute the block of code. It continues to execute as long as the specified condition is true.
- It's important to make sure that the condition eventually becomes false; otherwise, the loop will run indefinitely, resulting in an infinite loop.

While Loop Syntax:

The syntax of a while loop is straightforward:

```
while                                     (condition){  
  # Code to be executed while the condition is true  
}
```

Examples:

C++JavaPython3C#JavaScript



```
#include <iostream>
using namespace std;

int main()
{
    int count = 0;
    while (count < 5) {
        cout << count << "\n";
        count += 1;
    }
    cout << endl;
    return 0;
}
```

### Do-While Loop in Programming:

- The do-while loop is similar to the while loop, but with one key difference: it guarantees that the block of code will execute at least once before checking the condition.
- This makes it useful when you want to ensure that a certain task is performed before evaluating a condition for continuation.
- The loop continues to execute as long as the specified condition is true after the first execution. It's crucial to ensure that the condition eventually becomes false to prevent the loop from running indefinitely, leading to an infinite loop.

### **Syntax of do...while Loop:**

```
do {
    // body of do-while loop
```

```
} while (condition);
```

Examples:

C++JavaPython3C#JavaScript



```
#include <iostream>
using namespace std;
```

```
int main()
{
    int count = 5;
    do {
        count += 1;
    } while (count < 5);
    cout << "Final value of count = " << count;
    return 0;
}
```

Difference between For, While and Do-While Loop in Programming:

| Feature | for Loop                                                                    | while Loop                  | do-while Loop                |
|---------|-----------------------------------------------------------------------------|-----------------------------|------------------------------|
| Syntax  | for           (initialization;<br>condition;<br>increment/decrement)<br>{ } | while<br>(condition)<br>{ } | do { } while<br>(condition); |

| Feature        | for Loop                                                               | while Loop                                                            | do-while Loop                                                                |
|----------------|------------------------------------------------------------------------|-----------------------------------------------------------------------|------------------------------------------------------------------------------|
| Initialization | Declared within the loop structure and executed once at the beginning. | Declared outside the loop; should be done explicitly before the loop. | Declared outside the loop structure                                          |
| Condition      | Checked before each iteration.                                         | Checked before each iteration.                                        | Checked after each iteration.                                                |
| Update         | Executed after each iteration.                                         | Executed inside the loop; needs to be handled explicitly.             | Executed inside the loop; needs to be handled explicitly.                    |
| Use Cases      | Suitable for a known number of iterations or when looping over ranges. | Useful when the number of iterations is not known in advance          | Useful when the loop block must be executed at least once, regardless of the |

| Feature                         | for Loop                  | while Loop                                                     | do-while Loop                                                  |
|---------------------------------|---------------------------|----------------------------------------------------------------|----------------------------------------------------------------|
|                                 |                           | or based on a condition.                                       | initial condition.                                             |
| Initialization and Update Scope | Limited to the loop body. | Scope extends beyond the loop; needs to be handled explicitly. | Scope extends beyond the loop; needs to be handled explicitly. |

### • Loop control statement :

In C programming, break, continue, and goto are jump statements that control the flow of execution. break terminates a loop or switch statement, continue skips the current iteration of a loop and proceeds to the next, and goto transfers control to a specific label within the same function.

break Statement:

The break statement is used to exit a loop or switch statement prematurely.

C

#include

<stdio.h>

```

int main()
{
    int i;
    for (i = 1; i <= 10; i++)
    {
        if (i == 5)
            break;
    }
}

```

```

        break;
    }
    printf("%d", i);
}
printf("\n");
return 0;
}

```

In this example, the loop will only print numbers from 1 to 4. When *i* reaches 5, the break statement is executed, and control is transferred to the line immediately after the loop.

### continue Statement:

The continue statement skips the current iteration of a loop and proceeds to the next iteration.

C

```

#include <stdio.h>

int main()
{
    int i;
    for (i = 1; i <= 10; i++)
    {
        if (i == 5)
            continue;
    }
    printf("%d", i);
}
printf("\n");
return 0;
}

```

### goto Statement:

The goto statement transfers control to a label within the same function.

C

```

#include <stdio.h>

int main()
{
    int i;
    for (i = 1; i <= 10; i++) {
        if (i == 5) {
            goto label;
        }
        printf("%d", i);
    }
    label:
    printf("\n");
    return 0;
}

```

- **Function in C :**

A **function in C** is a set of statements that, when called, perform some specific tasks. It is the basic building block of a C program that provides modularity and code reusability. They are also called subroutines or procedures in other languages.

### Function Definition:

A function definition informs the compiler about the function's name, its return type, and what it does. It is compulsory to define a function before it can be called.

```

return_type name () {
    // Body of function
};

```

where,

- **return\_type:** type of value the function return.
- **name:** Name of the function
- **Body of function:** Statements inside curly brackets { } are executed when function call.

### Example:

```

void hello(){
    printf("GeeksforGeeks");
}

```

```
}
```

In the above code, **hello** is the name assigned to the function and its return type is **void**.

### Function Call

After defining a function, you can use it anywhere in the program by simply calling it with its name followed by **parentheses ()**.

### Example:



```
#include <stdio.h>
```

```
// Function definition
```

```
void hello() {  
    printf("sunil");  
}
```

```
int main() {
```

```
    // Function call
```

```
    hello();  
    return 0;
```

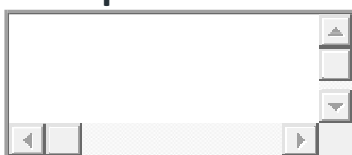
```
}
```

### Return Type of Functions:

A function can return a value to its caller as a result. It is called the **return value**, and the type of this value is called **return type** of the function. The function only returns one value, and the value time is the same as the function's return type.

The **return keyword** is used to return some values from the function.

### Example:



```
#include <stdio.h>
```



```

int getThree(){
    int n = 3;
    return n;
}

int main() {

    // Print the value that
    // is return by getThree()
    // function
    printf("%d", getThree());
    return 0;
}

```

### Function Parameters:

A function can be provided some values by its caller. These values are called **arguments** and are provided at the time of function call. In the function definition, we use the placeholder variables inside the parentheses () to receive these values. These placeholders are called **parameters**.

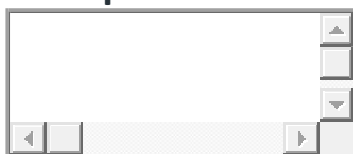
```

// Parameters in the function definition
return_type func(type1 name1, type2 name2, ...){
    // ....
}

```

**name1, name2, ....** are the names given to the parameters of the function. They will be referred with the same name inside the function.

### Example:



```
#include <stdio.h>
```

```
// Defining a function that
```

```
// print square of given number
void printVal(int num, float real){
    printf("%d %f\n", num, real);
}

int main() {
    int a = 3;

    // Call the printVal function and pass
    // desired values
    printVal(a, 1.5);
    return 0;
}
```

It is to be noted that the **number and types of argument passed in the function call should match the number and types of parameters defined in the function definition. Even the sequence should be same.**

There are also other different ways to pass arguments to the function. Refer to this article to know more – [Parameter Passing Techniques in C](#)

### Forward Declaration:

In C, a function must be defined before it is called, or the compiler will generate an error. To prevent this, we can declare the function ahead of the call and definition, providing the compiler with its name, return type, and parameters. This is known as a **forward declaration**.

```
return_type name();
```

The above statement tells the compiler about a **function's name and return type**. Mentioning parameters in the declaration is optional but valid.

```
return_type name(type1, type2, ...);
```

This type of function declaration is known as a **function prototype** or **function signature**. If it appears before the function call, we can define the function anywhere in the program.

**Note:** *Function declarations are required, but since the function definition includes the declaration, it's not necessary to declare it separately when the function is defined at the beginning.*

### Local Variables:

Variables declared inside a function are called **local variables** because they are only accessible within that function. We cannot access them outside the function.

#### Example:

```
#include <stdio.h>

int getThree(){
    int n = 3;
    return n;
}

int getThreeDummy(){

    // Try to access a variable
    // of another function
    return n;
}

int main() {

    printf("%d", getThreeDummy());
    return 0;
}
```

In the above program, we attempt to access a variable from another function, but the compiler gives an error saying that the variable is

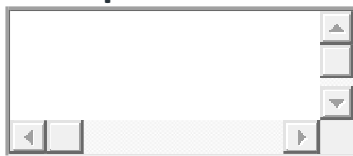
undefined. This means the local variable is not accessible outside the function.

In contrast, global variables can be accessed anywhere in the program, including inside functions.

### Library Functions

These functions are part of C libraries, which are pre-defined and perform specific tasks. By using them, you can save time and avoid errors with different inputs, as these functions are already tested and optimized for use.

### Example:



```
#include <stdio.h>
#include <math.h>
```

```
int main() {
    int n = 3;
    int m = 6;

    // Using fmax() from math.h to find the maximum
    printf("%.0f", fmax(n, m));

    return 0;
}
```

### Main Function in C:

If you read the article carefully, you will notice that we use the main function in every program example. In C programming, there is an entry point where the program starts its execution. So main() function is nothing it is an entry point of a program. The return value of main function indicates a program successfully or unsuccessfully execute or not.

## Memory Management of C Functions:

When a function is called, memory for its variables and other data is allocated in a separate block in a stack called a **stack frame**. The stack in which it is created is called **function call stack**. When the function completes its execution, its stack frame is deleted from the stack, freeing up the memory.

Refer to this article to know more - [Function Call Stack in C](#)

## Recursion:

Recursion is a technique where a problem is solved by breaking it into smaller subproblems. It involves a function calling itself, known as a recursive call, and the function is called a recursive function.

## • Array in C

An **array** in C is a **collection of elements** that are **of the same data type** and are stored **contiguously in memory**. Arrays allow you to store multiple values using a **single variable name**, and access them using **indices**.

## Declaration and Initialization of Arrays

### Syntax:

```
data_type array_name[size];
```

### Example:

```
int numbers[5]; // declares an array of 5 integers
```

You can initialize it at the time of declaration:

```
int numbers[5] = {10, 20, 30, 40, 50};
```

### Accessing elements:

```
printf("%d", numbers[2]); // prints 30 (arrays are 0-indexed)
```

## 1. One-Dimensional Arrays

A **one-dimensional array** is like a list – it stores a single row of elements.

Example:

```
#include <stdio.h>
```

```
int main() {
```

```
    int marks[5] = {90, 85, 75, 88, 92};
```

```
    for(int i = 0; i < 5; i++) {
```

```
        printf("marks[%d] = %d\n", i, marks[i]);
```

```
    }
```

```
    return 0;
```

```
}
```

## 2. Multi-Dimensional Arrays

A **multi-dimensional array** is an array of arrays. The most common type is the **two-dimensional array**, which can be visualized as a **table (rows and columns)**.

Syntax:

```
data_type array_name[rows][columns];
```

Example (2D Array):

```
#include <stdio.h>
```

```
int main() {
```

```
    int matrix[2][3] = {
```

```
        {1, 2, 3},
```

```
        {4, 5, 6}
```

```
    };
```

```

for(int i = 0; i < 2; i++) {
    for(int j = 0; j < 3; j++) {
        printf("matrix[%d][%d] = %d\n", i, j, matrix[i][j]);
    }
}
return 0;
}

```

## Difference between One-Dimensional and Multi-Dimensional Arrays

| Feature       | One-Dimensional Array             | Multi-Dimensional Array                      |
|---------------|-----------------------------------|----------------------------------------------|
| Structure     | Single list of elements           | Table-like (arrays of arrays)                |
| Declaration   | int arr[5];                       | int arr[2][3];                               |
| Access        | arr[0]                            | arr[0][0]                                    |
| Use Case      | Storing linear data (e.g., marks) | Storing matrix-like data (e.g., grid, table) |
| Visualization | 1 row                             | Multiple rows and columns                    |

## • Pointers in C

What Are Pointers in C?

A **pointer** in C is a **variable that stores the memory address** of another variable. Instead of holding a data value directly, a pointer holds the **location in memory** where the value is stored.

## Why Are Pointers Important in C?

Pointers are fundamental in C for several reasons:

| Reason                              | Explanation                                                                       |
|-------------------------------------|-----------------------------------------------------------------------------------|
| <b>1. Efficient Memory Access</b>   | Allows direct access and manipulation of memory.                                  |
| <b>2. Function Arguments</b>        | Enables <b>pass-by-reference</b> , allowing functions to modify actual variables. |
| <b>3. Dynamic Memory Allocation</b> | Used with malloc, calloc, etc., for runtime memory management.                    |
| <b>4. Data Structures</b>           | Crucial for implementing <b>linked lists, trees, graphs</b> , etc.                |
| <b>5. Arrays and Strings</b>        | Arrays and pointers are closely related and interchangeable in many ways.         |

## Declaration and Initialization of Pointers

### Declaration

```
data_type *pointer_name;
```

\* indicates that it is a pointer.

The pointer type must match the type of data it will point to.

Example:

```
int x = 10;
```



```
int *ptr;    // declaration
ptr = &x;    // initialization with address of x
```

&x gives the **address** of variable x.

\*ptr gives the **value** at that address (dereferencing).

### Basic Pointer Example

```
#include <stdio.h>
```

```
int main() {
    int x = 25;
    int *p = &x;

    printf("Value of x: %d\n", x);
    printf("Address of x: %p\n", &x);
    printf("Value of p: %p\n", p);
    printf("Value pointed by p: %d\n", *p);

    return 0;
}
```

### Key Pointer Operators

#### Operator Meaning

|   |                       |
|---|-----------------------|
| & | "Address of" operator |
|---|-----------------------|

## Operator Meaning

\* "Dereference" operator – gets value at address

## Common Pointer Types

| Pointer Type                      | Example                     |
|-----------------------------------|-----------------------------|
| Pointer to int                    | <code>int *p;</code>        |
| Pointer to float                  | <code>float *f;</code>      |
| Pointer to char (used in strings) | <code>char *str;</code>     |
| Null pointer                      | <code>int *p = NULL;</code> |
| Void pointer (generic)            | <code>void *ptr;</code>     |

- **Strings in C**

### 1. `strlen()` – String Length

**Purpose:**

Returns the **length of a string**, excluding the null character.

**Syntax:**

```
size_t strlen(const char *str);
```

**Example:**

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {
    char name[] = "Jonny";
    printf("Length of name: %lu\n", strlen(name)); // Output: 5
    return 0;
}
```

### **Use**

### **Case:**

Used when you need to know the size of a string, e.g., input validation or memory allocation.

## **2. strcpy() – String Copy**

### **Purpose:**

Copies a string from source to destination.

### **Syntax:**

```
char *strcpy(char *dest, const char *src);
```

### **Example:**

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {
    char src[] = "Hello";
    char dest[20];
    strcpy(dest, src);
    printf("Copied string: %s\n", dest); // Output: Hello
    return 0;
}
```

```
}
```

**Use****Case:**

When you need to duplicate a string or copy one into a new variable.

### 3. strcat() – String Concatenation

**Purpose:**

Appends one string to the end of another.

**Syntax:**

```
char *strcat(char *dest, const char *src);
```

**Example:**

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {
```

```
    char greeting[50] = "Hello, ";
```

```
    char name[] = "World!";
```

```
    strcat(greeting, name);
```

```
    printf("Concatenated string: %s\n", greeting); // Output: Hello, World!
```

```
    return 0;
```

```
}
```

**Use****Case:**

Used to join multiple strings (e.g., building full names, messages, file paths).

#### 4. strcmp() – String Compare

**Purpose:**

Compares two strings **lexicographically**.

**Syntax:**

```
int strcmp(const char *str1, const char *str2);
```

**Return Values:**

0 → Strings are equal

< 0 → str1 < str2

> 0 → str1 > str2

**Example:**

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {  
    char a[] = "Apple";  
    char b[] = "Banana";  
  
    int result = strcmp(a, b);  
    if(result == 0)  
        printf("Strings are equal\n");  
    else if(result < 0)  
        printf("a is less than b\n");  
    else  
        printf("a is greater than b\n");  
}
```

```
    return 0;
}
```

**Use****Case:**

Used in sorting strings, searching, or conditional checks (e.g., user login).

**5. strchr() – Character Search****Purpose:**

Finds the **first occurrence** of a character in a string.

**Syntax:**

```
char *strchr(const char *str, int ch);
```

**Example:**

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {
    char str[] = "Programming";
    char *ptr = strchr(str, 'g');

    if(ptr != NULL)
        printf("First 'g' found at position: %ld\n", ptr - str); // Output: 3
    else
        printf("'g' not found\n");

    return 0;
}
```

**Use****Case:**

Helpful in searching characters in strings (e.g., finding '@' in an email).

**Summary Table**

| Function Purpose |                          | Example Use Case                     |
|------------------|--------------------------|--------------------------------------|
| strlen()         | Find length of string    | Input validation, loop limits        |
| strcpy()         | Copy string              | Duplicating input                    |
| strcat()         | Concatenate strings      | Full name creation, message building |
| strcmp()         | Compare two strings      | Sorting, searching, authentication   |
| strchr()         | Find character in string | Character search, parsing            |

- **Structures in C**

## Concept of Structures in C

In C, a **structure (struct)** is a **user-defined data type** that allows grouping variables of **different data types** under a single name. It is used to represent a **record**, such as a student, employee, book, etc.

### Why Use Structures?

Structures help in:

Grouping related data together

Creating complex data types (like records)

Organizing code better for real-world entities

### Declaring a Structure

#### Syntax:

```
struct StructureName {  
    data_type member1;  
    data_type member2;  
    ...  
};
```

#### Example:

```
struct Student {  
    int rollNo;  
    char name[50];  
    float marks;  
};
```

### Declaring Structure Variables

You can declare structure variables in two ways:



### **1. Separately:**

```
struct Student s1;
```

### **2. At the time of structure definition:**

```
struct Student {  
    int rollNo;  
    char name[50];  
    float marks;  
} s1, s2;
```

### **Initializing Structure Members**

#### **Method 1: Initialization at declaration**

```
struct Student s1 = {101, "Anil", 92.5};
```

#### **Method 2: Assigning values individually**

```
strcpy(s1.name, "Anil"); // use strcpy for strings
```

```
s1.rollNo = 101;
```

```
s1.marks = 92.5;
```

### **Accessing Structure Members**

Use the **dot (.)** operator with the structure variable:

```
printf("Roll No: %d\n", s1.rollNo);
```

```
printf("Name: %s\n", s1.name);
```

```
printf("Marks: %.2f\n", s1.marks);
```

### **Complete Example**

```
#include <stdio.h>
```

```
#include <string.h>
```

```
struct Student {  
    int rollNo;  
    char name[50];  
    float marks;  
};  
  
int main() {  
    struct Student s1;  
  
    s1.rollNo = 101;  
    strcpy(s1.name, "Anil"); // Use strcpy for strings  
    s1.marks = 92.5;  
  
    printf("Student Details:\n");  
    printf("Roll No: %d\n", s1.rollNo);  
    printf("Name: %s\n", s1.name);  
    printf("Marks: %.2f\n", s1.marks);  
  
    return 0;  
}
```

## **Summary Table**

| Task               | Syntax / Example                                            |
|--------------------|-------------------------------------------------------------|
| Define a structure | <code>struct Student { int rollNo; char name[20]; };</code> |
| Declare a variable | <code>struct Student s1;</code>                             |
| Initialize values  | <code>s1.rollNo = 1; strcpy(s1.name, "Raj");</code>         |
| Access members     | <code>s1.name, s1.marks</code>                              |

## • File Handling in C

### Importance of File Handling in C

File handling in C is essential for:

**Storing data permanently** (unlike variables, which lose values after program ends)

**Reading and writing data** from/to files (e.g., logs, reports, configuration)

**Data sharing** between programs

Creating programs like **text editors, databases, file processors**, etc.

C provides a set of **functions in <stdio.h>** for file handling.

### Basic File Operations

#### Operation    Function Used

Open file    `fopen()`

Close file    `fclose()`

## **Operation    Function Used**

Read file    `fgetc()`, `fgets()`, `fread()`

Write file    `fputc()`, `fputs()`, `fprintf()`, `fwrite()`

### **1. Opening a File – `fopen()`**

#### **Syntax:**

`FILE *fp;`

`fp = fopen("filename.txt", "mode");`

#### **Common Modes:**

| <b>Mode</b> | <b>Description</b> |
|-------------|--------------------|
|-------------|--------------------|

|     |                        |
|-----|------------------------|
| "r" | Read (file must exist) |
|-----|------------------------|

|     |                                   |
|-----|-----------------------------------|
| "w" | Write (creates new or overwrites) |
|-----|-----------------------------------|

|     |                                |
|-----|--------------------------------|
| "a" | Append (adds to end if exists) |
|-----|--------------------------------|

|      |                                |
|------|--------------------------------|
| "r+" | Read + Write (file must exist) |
|------|--------------------------------|

|      |                                      |
|------|--------------------------------------|
| "w+" | Read + Write (creates or overwrites) |
|------|--------------------------------------|

|      |               |
|------|---------------|
| "a+" | Read + Append |
|------|---------------|

### **2. Writing to a File**

Example: `fprintf()` and `fputs()`

`#include <stdio.h>`

```
int main() {  
    FILE *fp = fopen("data.txt", "w");  
  
    if (fp == NULL) {  
        printf("File can't be opened\n");  
        return 1;  
    }  
  
    fprintf(fp, "Name: Jonny\n");  
    fputs("Marks: 85\n", fp);  
  
    fclose(fp);  
    return 0;  
}
```

### 3. Reading from a File

Example: fscanf() and fgets()

```
#include <stdio.h>
```

```
int main() {  
    char buffer[100];  
    FILE *fp = fopen("data.txt", "r");  
  
    if (fp == NULL) {  
        printf("File not found\n");  
    }  
}
```

```

        return 1;
    }

    while (fgets(buffer, 100, fp) != NULL) {
        printf("%s", buffer); // Prints each line
    }

    fclose(fp);
    return 0;
}

```

#### **4. Closing a File – fclose()**

Always close the file after finishing operations to:

Free memory

Avoid corruption

Ensure data is saved

```
fclose(fp);
```

#### **Other Useful File Functions**

| <b>Function</b> | <b>Description</b>   |
|-----------------|----------------------|
| fgetc(fp)       | Reads one character  |
| fputc(ch,fp)    | Writes one character |
| feof(fp)        | Checks end-of-file   |
| fread()         | Reads binary data    |

| Function              | Description        |
|-----------------------|--------------------|
| <code>fwrite()</code> | Writes binary data |

### Summary Table

| Operation      | Function Example                                |
|----------------|-------------------------------------------------|
| Open a file    | <code>FILE *fp = fopen("file.txt", "r");</code> |
| Write to file  | <code>fprintf(fp, "Hello");</code>              |
| Read from file | <code>fgets(buffer, size, fp);</code>           |
| Close the file | <code>fclose(fp);</code>                        |